

# ORGNZ-01: Introduction to Organizing Data in Grail

---

**Series:** ORGNZ | **Notebook:** 1 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

## Overview

---

Dynatrace Grail organizes data in **buckets**, **tables**, and **views** to ensure efficient storage, flexible access, and scalable querying. Understanding how to organize your data is fundamental to achieving optimal query performance, compliance requirements, and access control.

## Prerequisites

---

| Requirement | Details |  
|---

---

## Table of Contents

---

1. [Why Organize Data?](#)
  2. [The Grail Data Model](#)
  3. [Three Pillars of Data Organization](#)
  4. [When to Use Each Mechanism](#)
  5. [Permission Levels in Grail](#)
  6. [Exploring Your Data Organization](#)
  7. [Organization Decision Framework](#)
-

-----|-----|

| **Dynatrace Environment** | SaaS environment with Grail enabled |

| **Permissions** | `storage:buckets:read` for viewing data organization |

| **Knowledge** | Basic familiarity with Dynatrace and DQL |

## Learning Objectives

---

By the end of this notebook, you will:

- Understand why data organization matters in Grail
- Know the three pillars of data organization: buckets, segments, and security
- Recognize when to use each organization mechanism
- Understand the Grail data model structure

## Why Organize Data?

---

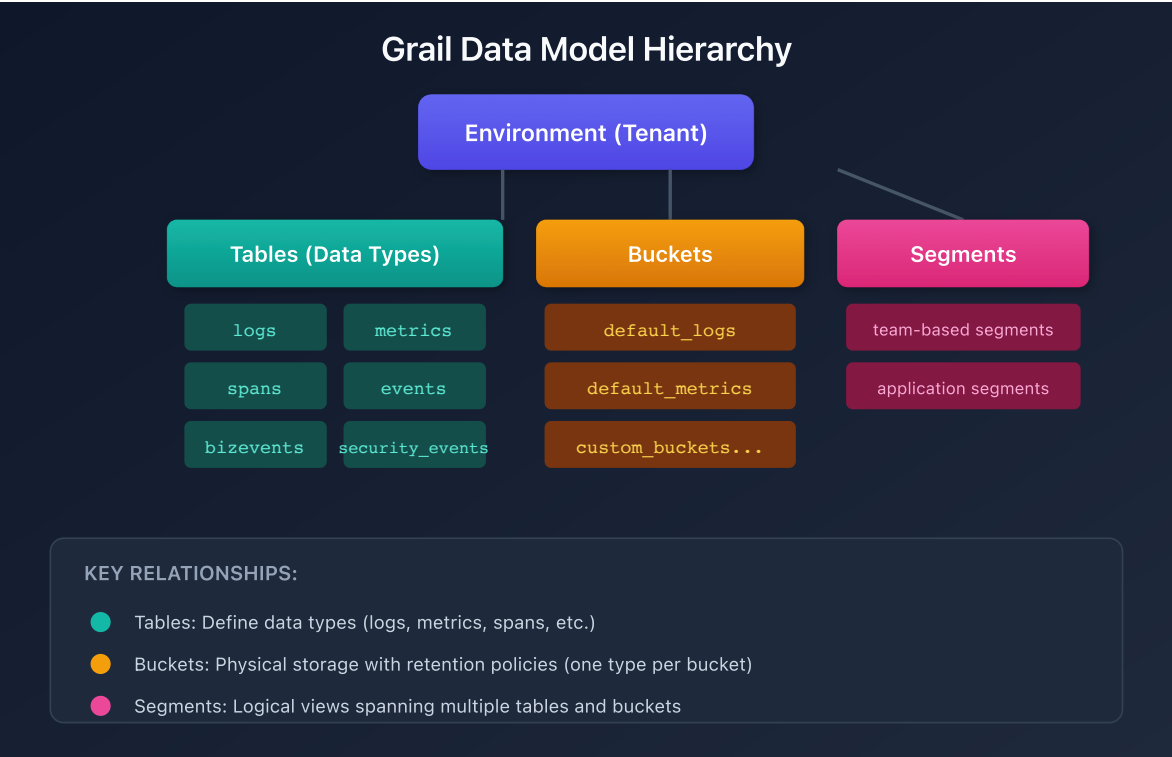
Effective data organization in Grail enables:

| Benefit                    | Description                                               |
|----------------------------|-----------------------------------------------------------|
| <b>Query Performance</b>   | Reduce query execution time by limiting data scope        |
| <b>Cost Control</b>        | Transparent GiB/day pricing with bucket-level attribution |
| <b>Access Control</b>      | Fine-grained permissions from bucket to field level       |
| <b>Compliance</b>          | Meet retention and data residency requirements            |
| <b>Operational Clarity</b> | Teams see only relevant data for faster troubleshooting   |

## The Grail Data Model

---

Grail organizes data hierarchically with tables (data types), buckets (storage containers), and segments (logical views):



# Three Pillars of Data Organization

## 1. Buckets (Physical Organization)

Buckets are logical storage containers where records are stored:

| Aspect    | Description                                      |
|-----------|--------------------------------------------------|
| Purpose   | Physical data partitioning and retention control |
| Scope     | One data type per bucket (logs, metrics, etc.)   |
| Retention | 1 day to 10 years per bucket                     |
| Access    | IAM policies can grant bucket-level access       |
| Cost      | Enable precise cost attribution by team/LOB      |

## 2. Segments (Logical Organization)

Segments provide dynamic, cross-signal data views:

| Aspect             | Description                                     |
|--------------------|-------------------------------------------------|
| <b>Purpose</b>     | Real-time filtering without physical separation |
| <b>Scope</b>       | Spans multiple data types and buckets           |
| <b>Flexibility</b> | Rule-based, can use variables and relationships |
| <b>Access</b>      | Governed by existing access controls            |
| <b>Cost</b>        | No direct storage cost impact                   |

### 3. Security Context (Access Control)

Security context enables fine-grained permissions:

| Aspect             | Description                                           |
|--------------------|-------------------------------------------------------|
| <b>Purpose</b>     | Record-level access control                           |
| <b>Mechanism</b>   | <code>dt.security_context</code> attribute on records |
| <b>Flexibility</b> | Custom values, hierarchical encoding                  |
| <b>Enforcement</b> | IAM policies filter at query time                     |

## When to Use Each Mechanism

| Need                        | Solution                | Rationale                              |
|-----------------------------|-------------------------|----------------------------------------|
| Different retention periods | <b>Buckets</b>          | Retention is set per bucket            |
| Cost attribution by team    | <b>Buckets</b>          | Direct GiB/day billing visibility      |
| Broad team isolation        | <b>Buckets</b>          | Simple bucket-level IAM policies       |
| Dynamic data views          | <b>Segments</b>         | Filter across data types at query time |
| Application-centric views   | <b>Segments</b>         | Group related entities and signals     |
| Record-level access control | <b>Security Context</b> | Fine-grained ABAC                      |
| Multi-team shared buckets   | <b>Security Context</b> | Filter records within same bucket      |

## Permission Levels in Grail

Grail supports permissions at multiple levels:

| Level         | Granularity                     | Example Use Case                           |
|---------------|---------------------------------|--------------------------------------------|
| <b>Bucket</b> | All records in a bucket         | Team owns entire bucket                    |
| <b>Table</b>  | All records of a data type      | Access to all logs                         |
| <b>Record</b> | Individual records by attribute | By host group, namespace, security context |
| <b>Field</b>  | Specific fields on records      | Mask sensitive fields                      |

**Important:** Without permissions, users cannot query data from Grail. Permissions must be explicitly granted.

## Exploring Your Data Organization

```
// List all buckets in your environment
fetch dt.system.buckets
| fields name, display_name, dt.system.table, retention_days,
estimated_uncompressed_bytes
| sort dt.system.table asc, name asc
```

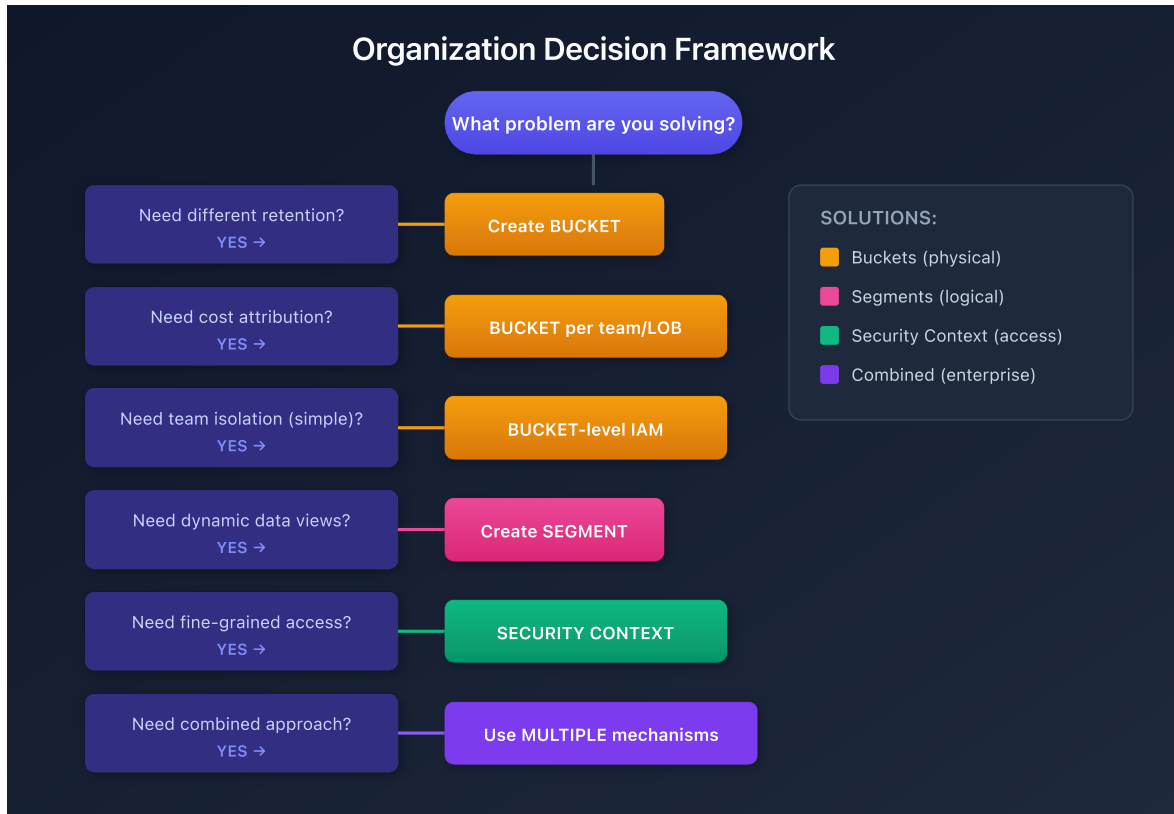
```
// Summarize data volume by bucket
fetch logs
| summarize recordCount = count(), by:{dt.system.bucket}
| sort recordCount desc
```

```
// Check for records with security context
fetch logs
| filter isNotNull(dt.security_context)
| summarize count = count(), by:{dt.security_context}
| sort count desc
| limit 20
```

# Organization Decision Framework

---

Use this framework to decide how to organize your data:



## Series Overview

---

This series covers data organization in depth:

| Notebook | Topic                | Focus                                        |
|----------|----------------------|----------------------------------------------|
| ORGNZ-01 | Introduction         | Why and how to organize data                 |
| ORGNZ-02 | Grail Buckets        | Bucket fundamentals and limits               |
| ORGNZ-03 | Bucket Strategy      | Naming, retention, design patterns           |
| ORGNZ-04 | Permissions Overview | Permission levels and policy management      |
| ORGNZ-05 | Bucket-Level Access  | IAM policies for bucket access               |
| ORGNZ-06 | Security Context     | Fine-grained access with dt.security_context |
| ORGNZ-07 | Advanced Permissions | Record-level and field-based access          |
| ORGNZ-08 | Grail Segments       | Dynamic data organization                    |
| ORGNZ-09 | Enterprise Patterns  | Combined approaches and best practices       |

## Next Steps

---

Continue with the ORGNZ series:

- **ORGNZ-02:** Understanding Grail Buckets

## References

---

- [Organize data](#)
- [Permissions in Grail](#)
- [Grail data model](#)

---

*This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*